

SmartGrid Lambda Pipeline

Project Handover and Orientation Guide

EC8203 Applied Big Data Engineering
Mini Project Assessment

Use Case 3 — Smart Grid Energy Monitoring and Billing
Lambda Architecture

github.com/Raviyaa27/smartgrid-lambda-pipeline
25 September 2026

How to read this document

This guide brings everyone on the team up to the same level of understanding of the project. It assumes no prior knowledge of what has been built. By the end you should be able to explain what the system does, why it is designed the way it is, run it on your own laptop, and answer questions about it in the demo or viva.

Sections 1 to 4 are the ones everybody must read — they cover the assignment, the use case and the core architectural idea. These are the parts the marking rubric weights most heavily, and they are the parts we will be questioned on. Sections 5 to 7 explain the tools and the code, and get the system running on your machine. Sections 8 to 10 are for revision and reference.

The single most important thing to take away

The assessment awards 20 marks out of 100 for the argument about **why we chose Lambda architecture over Kappa**, and a further 15 for the written report. That is 35 of 100 marks for reasoning rather than code. Section 4 of this guide is that argument in plain language. Every one of us needs to be able to make it without notes.

Section	What it covers	Priority
1. The assignment	What we must deliver and how it is marked	Everyone
2. The use case	The business problem we chose to solve	Everyone
3. Lambda architecture	The core idea, explained from scratch	Everyone
4. Lambda vs Kappa	Why we chose it — the 20-mark argument	Everyone
5. The technology stack	What each tool does and why it is there	Everyone
6. What is built so far	Progress to date, in detail	Everyone
7. Running it yourself	Step-by-step setup on your laptop	Everyone
8. Viva preparation	Likely questions with model answers	Revision
9. What remains	The roadmap for the rest of the build	Reference
10. Glossary	Plain-English definitions of the jargon	Reference

1. The assignment

What we have to build

We must design and implement a complete data platform that ingests information from two different kinds of source, processes it, and produces a consolidated report or dashboard that answers a real business question.

The two sources are deliberately different in character. One is a **streaming source** that emits events continuously, every few seconds. The other is a **daily batch source** that drops a single file once per day. Handling both properly, and joining them together meaningfully, is the heart of the exercise.

Critically, the brief does not tell us which architecture to use. It requires us to choose between **Lambda** and **Kappa**, and to defend that choice in writing. The choice itself is not marked right or wrong; the quality of the argument is what earns the marks.

How it is marked

Criterion	Marks	What this really means
Architecture decision and justification	20	Can we argue for Lambda over Kappa using the use case's latency, replay, cost and consistency needs? Do we discuss trade-offs honestly?
Report	15	Clear diagrams, clear explanation of decisions, presentation of results, honesty about limitations. 8 to 15 pages.
Data ingestion implementation	15	Are the simulated sources correct and robust?
Processing layer implementation	15	Is the transformation logic correct, and consistent with the architecture we declared?
Technology stack justification	10	Is each tool justified against our use case, not chosen because it is popular?
Storage and serving layer	10	Is the serving layer correctly implemented?
Observability	10	Logging, metrics and tracing good enough to detect and diagnose failures.
Code quality and documentation	5	Readable, modular, with a README and reproducible setup.

Where the easy marks are

Observability is worth 10 marks and is the criterion most teams under-deliver on. Most submissions add some logging and stop, scoring around half. We are building structured logging, metrics, end-to-end tracing and five alert rules — and in the demo we will deliberately break the pipeline so an alert can be seen firing. An alert that is *demonstrated* is worth far more than one that is merely configured.

What we must hand in

- **The code** — simulated sources, ingestion, processing, storage and serving layers, observability configuration, a README, and tests.
- **A report** of 8 to 15 pages, submitted as a PDF.
- **A demo** — either a 5 to 10 minute video, or a live demonstration showing the pipeline running end to end together with the observability results.
- **A statement of individual contributions**, since this is a group submission.

Read this part of the brief carefully

The brief states that using AI coding assistants for boilerplate is permitted, but that we **must be able to explain and defend every architectural decision and every line of core pipeline logic** in the viva or demo. Section 8 of this guide exists for exactly that reason. Please work through it before the demo.

2. The use case we chose

The brief offered three use cases and asked us to pick one: ride-hailing fleet operations, hospital patient vital-signs monitoring, or smart grid energy monitoring and billing. **We chose the smart grid use case.**

The scenario

A utility company wants real-time visibility into how much electricity the grid is carrying and how much of it is coming from rooftop solar, broken down by geographic zone. Separately, at the end of each day, it needs to work out what every household actually owes, using tariff and billing information that arrives once a day.

Our two data sources

	Streaming source	Daily batch source
What it is	Simulated smart meters in homes, reporting their readings continuously	A file dropped once per simulated day by the billing and weather systems
How often	Every couple of seconds, per meter	Once per simulated day
What it carries	Meter ID, household ID, electricity consumed, solar generated, grid zone, timestamp	Per household: tariff rate, billing tier, subsidy flag, fixed charge. Per zone: cloud cover and expected solar conditions
Why we need it	Tells us what is happening on the grid right now	Tells us what to charge for it

The business question

The question the whole system exists to answer

What is the current grid load and renewable contribution by zone, and what will each household's bill look like once daily tariff data is applied to their consumption?

Read that question again, because the entire architecture follows from one observation about it: **it is actually two questions, and they have completely different requirements.** Section 3 unpacks exactly what that means.

Why we picked this use case over the other two

All three options were viable. We chose the smart grid one because it gives us the strongest and most honest architectural argument, and the 20-mark criterion is precisely about the quality of that argument.

- **It contains two genuinely different correctness standards in one system.** Real-time grid monitoring can tolerate a missing reading; billing cannot tolerate any error at all. That contrast is the textbook case for Lambda, and here it is real rather than manufactured.
- **Billing must be recomputable years later.** Tariffs get revised retroactively by the regulator, meters go offline and backfill late, subsidy statuses change. Every one of those forces us to recalculate bills we already published. That requirement is what makes the Lambda argument decisive rather than marginal.
- **The billing calculation is genuinely complex.** Electricity is priced in blocks, not at a flat rate, so the calculation is real work. The brief explicitly penalises transformations that are just pass-through, so having a meaty calculation helps.
- **The join between the two sources is natural.** Meter readings join to tariff records on household ID. Nothing about it feels forced.

3. Lambda architecture, explained from scratch

The problem it solves

Go back to our business question. Split it in half and look at what each half actually needs.

	Half one: grid operations	Half two: billing
The question	How much load and solar, by zone, right now?	What does this household owe?
How fresh must the answer be?	Within about a minute	By 6am the next day is fine
How accurate must it be?	Roughly right is fine. One missing meter reading out of thousands changes nothing an operator would act on.	Exactly right. This is money. It is regulated, it appears on a customer's bill, and the customer can dispute it.
How long must we keep it?	Hours or days	Years
Who reads it?	Control-room dashboard and alerts	Billing system, the regulator, the customer
What happens if it is wrong?	An operator sees a slightly stale number	We overcharge someone, breach compliance, and issue refunds

These two columns are irreconcilable in a single processing path. Optimise for speed and billing becomes unreliable. Optimise for rigour and the control room gets yesterday's numbers. **Lambda architecture is the design that refuses to compromise and instead builds both.**

An everyday analogy

The shop till and the end-of-day accounts

Think of a shop. During the day the till display shows a running total of takings. It is instant and good enough to tell the manager whether it has been a busy morning. Nobody files it with the tax office.

After closing, the bookkeeper reconciles the day properly — against receipts, refunds, voids and the float. That figure is slower, authoritative, and it is the one that goes in the books. If a receipt turns up late, the bookkeeper redoes the day's accounts. Nobody redoes the till display.

The till display is our **speed layer**. The bookkeeper is our **batch layer**. Lambda architecture is simply having both, on purpose, and being clear about which number you are looking at.

The three layers

The speed layer

Reads meter readings off the stream as they arrive and keeps a running picture of each zone: total load, how much is solar, how many meters are reporting. It updates within seconds. It is allowed to be slightly wrong — if a reading arrives too late it may be missed, and that is an accepted trade-off for speed.

The batch layer

Once per day, recomputes the whole day from scratch using every reading that arrived, including the late ones the speed layer missed. It joins the readings to the daily tariff file and calculates the actual bills. It is slower but it is **authoritative**: whatever it produces is the official record.

The crucial property is that the batch layer reads from an **immutable archive** — a permanent, never-modified copy of every reading, filed by date. Because the input never changes, running the batch layer twice on the same day always produces the same answer. That is what makes a bill reproducible if a customer disputes it two years later.

The serving layer

The API and dashboards that people actually use. Its job is to combine the two layers and be honest about which one an answer came from.

The merge rule

This is the part most write-ups skip, and skipping it loses marks. Ours is explicit:

For zone metrics on date D:

```
if D is a PAST day  -> use the BATCH layer's figure    [ SETTLED    ]
if D is TODAY      -> use the SPEED layer's figure     [ PROVISIONAL ]
```

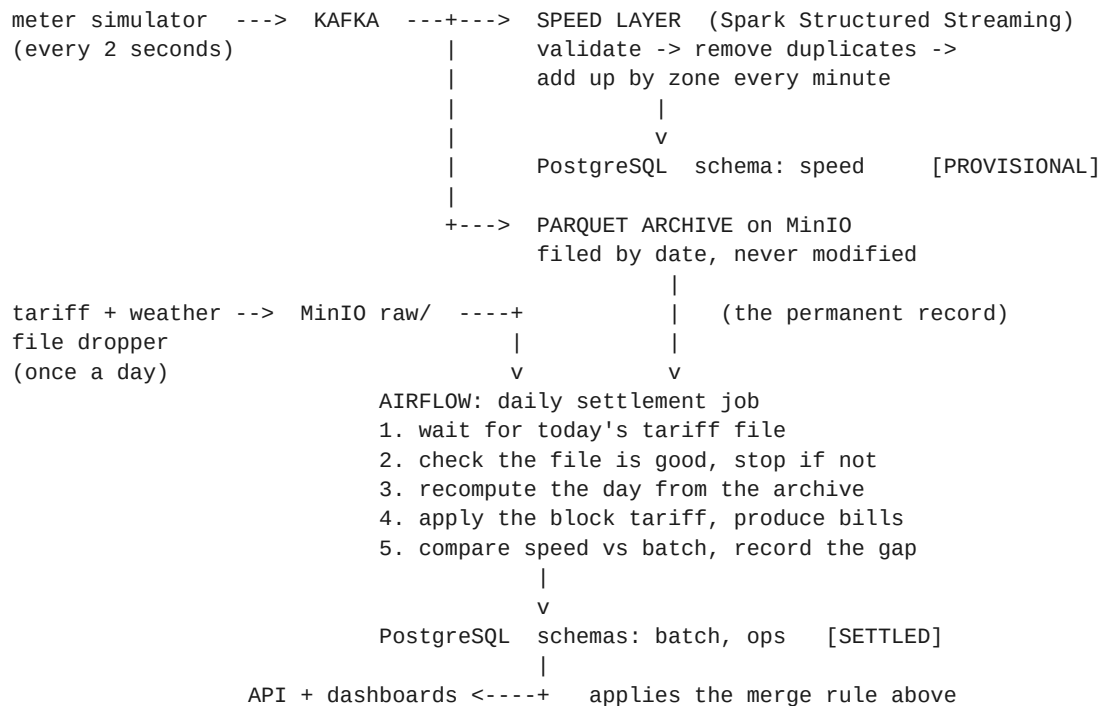
For a household bill:

```
always use the BATCH layer.
A provisional bill is NEVER published.
```

- **The batch layer always wins.** Once a day has been settled, the speed layer's numbers for that day are thrown away, not blended in. The speed layer is a stand-in, never a contributor to the official record.

- **There is never a provisional bill.** Billing tolerates zero approximation, so publishing an estimated charge would be a category error. The dashboard shows today's *consumption* live, but a *bill* only after settlement.
- **Every number on screen is labelled** PROVISIONAL or SETTLED. A number whose reliability is not stated will eventually be misused.

The whole system in one picture



Data flows down two paths from the same source. The speed path is fast and approximate; the batch path is slow and authoritative. They meet again in the serving layer.

4. Why Lambda and not Kappa

This section is the 20-mark argument. Please read it twice.

What Kappa is

Kappa architecture says: forget having two paths, that is needless duplication. Keep **everything** in the event log — in our case, Kafka — forever. Process it with a single streaming job. If you need to recompute history, rewind the log and replay it through the same job.

It is an elegant idea and it is the right answer for most streaming systems. One codebase, one engine, one thing to operate, and no possibility of two paths disagreeing because there is only one path. **We should be clear that Kappa is the sensible default, and that the burden of proof was on us to justify departing from it.**

The four things we compared

Axis	What our use case needs	How Kappa does	How Lambda does
Latency	Under a minute for the grid view; next-day for billing	Meets both comfortably. Kappa wins this one.	Speed layer covers the fast need, batch layer the slow one
Replay	Recompute any past day, for years back, when a tariff is revised	Requires keeping every reading in Kafka for years. Fixing one tariff row means replaying the entire log from that point forward	Re-run one job for one date against that date's archive file. Targeted and quick
Cost	A real utility produces roughly 1.4 billion readings a day	Around 736 TB of expensive message-broker disk, tripled for safety — about 2.2 PB, to support a handful of recalculations a year	Around 75 TB of cheap compressed archive storage. Roughly ten times less, on much cheaper hardware
Consistency	A bill must be exact and reprovable years later	Possible, but the serving database is being continuously rewritten by the stream, so proving what it said on a given date is hard	The batch layer is the declared official record. Reproducing an old bill just means running one job again

The decision in one sentence

We chose Lambda not because it is more powerful, but because **this use case contains two genuinely different correctness standards**, and Lambda lets each be met on its own terms instead of forcing one to compromise for the other.

The honest part — when we would be wrong

The rubric rewards honest discussion of trade-offs, so we say this explicitly in the report and we should say it in the viva too:

If billing were not in scope, Kappa would be the right answer

The batch layer exists solely to serve billing. If the requirement were only the real-time grid view, building a batch layer as well would be over-engineering, and Kappa would be the correct choice. Our decision record even lists the conditions under which the decision should be reversed.

Lambda's real weakness, and what we did about it

There is a well-known criticism of Lambda, made most clearly by Jay Kreps (one of the creators of Kafka) in a 2014 article called *Questioning the Lambda Architecture*. His argument is simple and correct: if you write the same business logic twice, once for the streaming path and once for the batch path, the two copies will eventually drift apart. And when they do, the symptom is horrible — a number silently changes when a day is settled, with no error message anywhere to explain why.

An examiner will raise this. Our answer is not 'we will be careful'. It is structural:

- **The rules are written down once, as data.** Every field the pipeline handles — its type, whether it is required, its valid range — is declared in a single table in the code. Nothing anywhere else defines what a valid reading is.
- **Both layers read that same table.** There are two validators, one for row-by-row Python and one for the bulk processing Spark uses, but neither contains any rules of its own. They both walk the same table. Adding a rule is a one-line edit that changes both at once.
- **Drift breaks the build.** There is an automated test that feeds the same data to both validators and asserts they reach identical verdicts. If they ever disagree, the test fails before the code can reach a customer's bill.
- **Both layers run on the same engine.** We use Spark for the streaming job *and* the daily job, so they share a language as well as a rule set.

We also measure how wrong the fast path is

Every night, the settlement job compares what the speed layer reported for each zone against what the batch layer finally settled, and records the difference. This turns 'the speed layer is approximate' from a hand-wave into an actual measured number that appears on the dashboard and can raise an alert if it drifts too far.

This is probably the single most convincing artefact in the whole project, because it shows we understand not just what Lambda is but what it costs.

5. The technology stack

Each tool is here for a specific reason tied to our use case. If asked in the viva why we used something, 'it is the industry standard' is a weak answer; the reasons below are the strong ones.

Tool	What it is, in one line	Why it is in our system
Apache Kafka	A durable log that producers write events into and consumers read from	Unlike a normal message queue, a log can be re-read. We keep 7 days — enough to recover from a crash. Long-term history is the archive's job, not Kafka's
Spark Structured Streaming	A processing engine that runs the same code over a stream or over files	Chosen over Storm mainly because the <i>same engine</i> runs both our layers, which halves the risk of them drifting apart. It also handles late-arriving data properly out of the box
Apache Airflow	A scheduler that runs jobs in a dependency order, with retries	Its 'backfill' feature — re-running a job for a past date — <i>is</i> our recalculation mechanism. That is the actual reason we chose it, not the scheduling
Parquet on MinIO	A compressed column-based file format, stored in an S3-style object store	This is the permanent archive. Filing by date means recomputing one day reads exactly one folder. MinIO speaks Amazon S3's language, so moving to the cloud would need no code changes

Tool	What it is, in one line	Why it is in our system
PostgreSQL	A standard relational database	What the dashboards and API read from. Our queries join households to tariffs to zones, which is exactly what relational databases are good at. Split into three areas: speed, batch and ops
FastAPI	A Python framework for building web APIs	Serves the real-time metrics endpoint the brief asks for, and applies the merge rule
Streamlit	A Python framework for building data dashboards quickly	The business-facing dashboard: bills, zone renewable mix, and the provisional-versus-settled comparison
Prometheus + Grafana	A metrics collector and a dashboard tool for it	The operations side of observability: throughput, lag, error rates, and the alert rules that fire when something breaks
Docker Compose	Runs all of the above as containers from one configuration file	Anyone can run the entire system with one command. The rubric rewards this directly under reproducibility

Why Spark runs inside Docker and not directly on Windows

Spark on Windows needs a Hadoop compatibility layer and a file called winutils.exe, and setting that up reliably is a notorious time sink. Running Spark inside a Linux container avoids the problem entirely and means the setup is identical on everyone's machine. You do not need Java installed.

6. What has been built so far

Three stages are complete. Everything is committed and pushed to the repository. The work was sequenced deliberately: infrastructure first, then the shared foundation code, then the written architecture decisions — all before any pipeline logic, so the design drives the build rather than being reverse-engineered from it afterwards.

Stage 1 — The infrastructure

A single configuration file brings up four services as containers. Two extra one-shot jobs run once at startup to create the Kafka topics and the storage buckets, then exit.

Service	What it does	Where to see it
Kafka	The ingestion log. 6 partitions, 7-day retention	localhost:9092
Kafka UI	A web page for inspecting topics and messages	localhost:8085
PostgreSQL	The serving database	localhost:5432
MinIO	The object store holding the archive	localhost:9001 (web console)

Two details in this configuration are worth knowing, because both encode architectural decisions rather than being arbitrary settings:

- **Kafka keeps only 7 days of data.** That is not laziness — it is the Lambda decision made visible in configuration. Kafka is transport and crash recovery here, not the permanent record. Under Kappa this number would have to be measured in years, which is the whole cost argument from Section 4.
- **The database is divided into three named areas** (speed, batch and ops), each with a comment in the database itself explaining what guarantees it carries. The provisional-versus-settled distinction is enforced by the structure of the database, so it cannot get lost on the way to whoever is reading it.

There is also an automated check, `scripts/smoke_test.py`, which confirms not just that the services are running but that they are correctly set up — the right topics with the right number of partitions, the right database areas, the right storage buckets. Run it whenever something seems wrong.

Stage 2 — The shared foundation code

A Python package called `smartgrid` that every later part of the pipeline builds on. Seven modules, backed by 46 automated tests.

Module	What it does
config	All settings in one typed place, read from a <code>.env</code> file. It also works out automatically whether the code is running on your laptop or inside a container, and picks the right addresses, so nothing downstream has to care
logging	Writes every log line as a single line of JSON so it can be read by a human or parsed by a machine. Each event gets a tracking ID that follows it all the way through the pipeline, which is what lets us trace one meter reading end to end
clock	The simulated clock. One simulated day is squeezed into 300 real seconds — 288 times faster than reality — so a whole week of grid operation fits into a 35-minute demo
domain	Builds the imaginary world: 200 households across 6 zones, 42% with solar panels. Generated from a fixed seed, so it is identical on everyone's machine. That is what makes the join between the two sources real rather than fictional
schemas	The single source of truth for what every data field is and what makes it valid. Both layers' validation is generated from these tables
transformations	The validation, cleaning and enrichment shared by both layers. This is the module that answers the criticism of Lambda described in Section 4
billing	The block-tariff calculation. Explained below, because it matters

How electricity billing actually works, and why it matters to us

Electricity is not charged at a flat rate per unit. It is charged in **blocks**: the first 60 units at one price, the next 60 at a higher price, and so on. A household that crosses a boundary pays the higher rate *only on the units above it*, not on everything.

Here is a worked example the system produces:

```
Household consumed      145.000 kWh
Solar generated         35.000 kWh
-----
Net drawn from the grid 110.000 kWh    <- only the NET position is billed

block  units kWh    rate    amount
```

0	60.000	12.00	720.00	first 60 units
1	50.000	27.75	1387.50	the next 50, at the higher rate

energy charge			2107.50	
fixed charge			300.00	
TOTAL PAYABLE			2407.50	LKR

Two things to notice, because both are worth marks. First, only the **net** position is billed — solar generation is subtracted before pricing, which is how net metering works in reality. Second, the 110 units are split across two price bands rather than multiplied by a single rate. The brief explicitly penalises transformations that are just pass-through, and this is our clearest evidence that ours are not.

A real bug we found, and why it is worth mentioning in the report

All money in the system is calculated using exact decimal arithmetic, never ordinary computer floating-point numbers. The reason is that a computer cannot represent 0.01 exactly in binary, so floating-point errors accumulate and bills stop adding up.

This paid off immediately. The monthly block limits have to be scaled down for daily billing, and the original code divided before multiplying. That turned a 30-unit monthly limit into 0.999999999999999999999999 units per day instead of exactly 1. A household consuming precisely the limit would have had one unit charged at the wrong, higher rate. Fixed by multiplying before dividing, with two tests added to pin the boundary.

Stage 3 — The architecture decision records

Seven documents, each capturing one decision: the context that forced it, what was chosen, what was rejected and why, what it costs us, and the conditions under which the decision should be reversed. They live in docs/adr/ in the repository.

Record	The decision	Decided on the grounds of
ADR-0001	Lambda architecture, not Kappa	Replay, cost and consistency. This is the 20-mark one
ADR-0002	Spark Structured Streaming, not Storm	Being able to use one engine for both layers
ADR-0003	Kafka topic layout and 7-day retention	Ordering per meter, and bounded storage cost
ADR-0004	One shared code module for both layers	Answering Lambda's known weakness structurally
ADR-0005	Parquet archive plus PostgreSQL serving	Two different access patterns, two stores
ADR-0006	Airflow for orchestration	Its backfill feature is our recalculation mechanism
ADR-0007	Simulated clock at 288 times speed	Fitting a week of operation into a demo

These were written **before** the pipeline code, on purpose. They become the architecture chapter of the final report almost word for word, and writing them first means the design led the implementation rather than being invented afterwards to justify it.

7. Running it on your own machine

What you need first

- **Docker Desktop**, running, with roughly 8 GB of RAM free.
- **Python 3.12** specifically. Newer versions do not yet have working installers for some of the libraries we use.
- **Git**.
- You do **not** need Java, Kafka, Spark or PostgreSQL installed. Everything runs in containers.

Step 1 — Get the code

```
git clone https://github.com/Raviyaa27/smartgrid-lambda-pipeline.git
cd smartgrid-lambda-pipeline
```

Step 2 — Create your settings file

The repository contains `.env.example`, which documents every setting. Copy it. The real `.env` is deliberately not committed, because passwords should never be in a repository.

```
copy .env.example .env
```

Step 3 — Start the infrastructure

```
docker compose up -d
```

The first run downloads several container images and takes a few minutes. Wait about 45 seconds after it finishes for Kafka to report itself healthy.

When you then run `docker compose ps`, two of the containers — the ones ending in `-init` — will show as **Exited (0)**. That is correct and not an error. They are one-shot setup jobs that create the Kafka topics and storage buckets and then finish. Exit code 0 means they succeeded.

Step 4 — Set up Python

```
py -3.12 -m venv .venv
.venv\Scripts\activate

pip install -r requirements-dev.txt
pip install -e .
```

If PowerShell refuses to run the activate script, run this once and try again: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Step 5 — Check everything works

```
python scripts/smoke_test.py
```

You should see:

```
smartgrid-lambda-pipeline :: infrastructure smoke test
-----
PASS  Kafka          1 broker(s), 'meter.readings.v1' x6, DLQ present
PASS  PostgreSQL     schemas=['batch', 'ops', 'speed'], airflow db ok
PASS  MinIO          buckets=['lake', 'raw']
```

All checks passed. Section 1 complete.

Then run the test suite and the demonstration script:

```
pytest                                expect: 46 passed

python scripts/section2_demo.py      shows the clock, the fleet, the JSON
                                     logs, validation, and a worked bill
```

Step 6 — Look around

What	Where	Login
Kafka topics and messages	http://localhost:8085	none
Storage buckets and files	http://localhost:9001	minioadmin / minioadmin123

Handy commands

To do this	Windows	Mac or Linux
Start everything	.\scripts\dev.ps1 up	make up
Stop everything	.\scripts\dev.ps1 down	make down
Wipe it and start clean	.\scripts\dev.ps1 reset	make reset
Watch the logs	.\scripts\dev.ps1 logs	make logs
Run the health check	.\scripts\dev.ps1 verify	make verify

If something goes wrong

What you see	What it means	What to do
port is already allocated	Something else on your machine is already using that port, commonly a local PostgreSQL on 5432	Stop the other program, or change the port in docker-compose.yml and .env
Smoke test says the database areas are missing	The database was created before the setup script existed. Those scripts only run on a brand-new empty database	.\scripts\dev.ps1 reset then start again
Kafka stuck as starting for over two minutes	It failed to start up properly, usually leftover state	docker compose logs kafka to look, then reset
pip install fails to build a package	You are on the wrong Python version	Check python --version says 3.12 inside the virtual environment; rebuild it with py -3.12
ModuleNotFoundError: smartgrid	The package was not installed into your environment	Run pip install -e . with the environment active
A container prints its own help text and stops	It was started without its arguments	Check the command formatting in docker-compose.yml

8. Questions we should each be able to answer

The brief warns that we must be able to defend every architectural decision and every line of core pipeline logic. These are the questions most likely to come up. The answers are short on purpose — say this much, then stop.

1. Why did you choose Lambda over Kappa?

Because the business question contains two halves with incompatible correctness standards. The real-time grid view tolerates approximation and needs sub-minute freshness. Billing tolerates none at all — it is regulated, disputable, and must be recomputable years later when a tariff is revised retroactively. Kappa would need years of Kafka retention to support that, roughly 2.2 petabytes of broker storage, for something exercised a handful of times a year. Lambda gets it from 75 terabytes of cheap archive.

2. When would Kappa have been the better choice?

If billing were not in scope. The batch layer exists only to serve billing. With just the real-time grid view, Kappa is correct and Lambda would be over-engineering. Our decision record lists that as a condition for reversing the decision.

3. Lambda means maintaining two copies of the same logic. How did you handle that?

That is Kreps' criticism and it is correct. We answered it structurally rather than with discipline. The validation rules are declared once as a data table. Both layers' validators read that same table and contain no rules of their own. An automated test feeds identical input to both and asserts they return identical verdicts, so any drift fails the build. Both layers also run on Spark, so they share a language as well as a rule set.

4. How do the two layers get combined when someone queries the system?

By an explicit merge rule. For any past date the batch figure is used and the speed figure is discarded — not averaged, discarded. For today, the speed figure is used and labelled provisional. For bills there is no merge at all: only the batch layer ever produces one, because a provisional bill would be a category error.

5. Why Spark and not Storm?

Mainly so that one engine runs both layers, which cuts the drift risk. Beyond that, Spark handles event-time windows and late-arriving data natively, which matters because meters go offline and backfill. Storm is at-least-once by default and has no event-time windowing, so we would have had to build both by hand.

6. Why Airflow? Could you not just use a scheduled script?

We did not choose it for scheduling. We chose it because its backfill feature is our recalculation mechanism. The job is parameterised by date, so recomputing a past day after a retroactive tariff change is a single command. A cron script would mean rebuilding that capability by hand, worse.

7. Why keep only 7 days in Kafka?

Because in our design Kafka is transport and crash recovery, not the permanent record. Seven days covers a consumer failing over a long weekend. The permanent record is the Parquet archive. That single setting is the Lambda decision expressed in configuration.

8. Why PostgreSQL and not Cassandra?

Our serving queries are relational and ad-hoc — households joined to tariffs joined to zones, aggregated differently per report. Cassandra needs the query pattern known up front and does not do ad-hoc joins. Its throughput advantage would not be exercised, because the serving layer holds aggregates, not raw readings. If we had to serve raw per-meter readings at full scale, Cassandra would be the right answer for that table.

9. How do you know your speed layer is accurate enough?

We measure it. Every night the settlement job compares what the speed layer reported per zone against what the batch layer settles, and records the difference. It is on the dashboard and it raises an alert if it drifts past a threshold.

10. Why is money handled as decimals rather than ordinary numbers?

Because binary floating point cannot represent 0.01 exactly, so errors accumulate and bills stop reconciling. That is unacceptable for a figure a customer can dispute. It caught a real bug: block limits were being computed as 0.9999... instead of exactly 1, which would have mispriced anyone landing on a boundary.

11. What is your simulated clock, and why does it matter?

One simulated day is 300 real seconds, 288 times faster than reality, so a week of operation fits into a 35-minute demo. What matters is the rule that goes with it: nothing in the pipeline reads the real clock to decide what day it is. All dates come from the simulated clock. That is what makes the archive partitions stable and the batch layer reproducible.

12. What would you do differently at production scale?

Adopt a table format such as Iceberg over the Parquet files, so restatement becomes transactional rather than a folder overwrite. Compile the validation rules into native Spark expressions instead of running them through Python, which is the throughput cost we knowingly accepted. Add a schema registry. Run Kafka with replication factor 3. And evaluate Flink, which paired with Iceberg could make the batch layer redundant altogether.

9. What still has to be built

Stage	What it delivers	Status
1. Infrastructure	Kafka, PostgreSQL, MinIO running under Docker	Done
2. Foundation code	Config, logging, clock, domain, schemas, billing	Done
3. Decision records	Seven ADRs, the architecture argument written down	Done
4. Streaming source	The meter simulator: a realistic daily solar curve, plus deliberate faults (bad values, duplicates, meters going silent) for the alerts to catch later	Next
5. Daily batch source	Drops the tariff and weather file once per simulated day, occasionally late or corrupt so the quality gate gets exercised	To do
6. Speed layer	The Spark streaming job: validate, remove duplicates, aggregate by zone, write to the database and to the archive	To do
7. Batch layer	The Airflow job: wait for the file, check it, recompute the day, apply the tariff, reconcile against the speed layer	To do
8. Serving API	FastAPI, applying the merge rule	To do
9. Dashboards	Streamlit for the business view, Grafana for operations	To do
10. Observability	Prometheus metrics, five alert rules, and a demonstrated failure	To do
11. Tests and polish	Integration tests, one-command demo, final README	To do
12. Report	The 8 to 15 page PDF, built from the decision records	To do
13. Demo video	5 to 10 minutes, end to end, including an alert firing	To do

Where help is most useful

If you want to pick something up, the highest-value contributions are not more code:

- **Read the decision records and challenge them.** If an argument does not convince you, it will not convince an examiner either. Finding the weak spot now is worth more than another feature.
- **Work through Section 8 of this guide.** Any of us could be asked any of those questions.
- **Draw the diagrams properly.** The report needs a system diagram, a data-flow diagram showing both paths meeting at the merge point, and a sequence diagram of a late reading being recalculated. Clear diagrams carry real weight in the 15 report marks.
- **Run the setup on a different machine and note where these instructions are wrong.** Reproducibility is directly assessed, and the only way to test it is for someone other than the author to follow the steps.

10. Glossary

Term	What it means
Batch layer	The slow, thorough path that recomputes a whole day from the permanent archive. Its output is the official record
Speed layer	The fast path that processes events as they arrive. Approximate, and always superseded by the batch layer
Serving layer	The API and dashboards people actually use, which combine the other two
Lambda architecture	Having a speed layer and a batch layer on purpose, each serving a different correctness requirement
Kappa architecture	The alternative: one streaming path only, with history recovered by replaying the event log
Replay / backfill / restatement	Recomputing data for a past date. Needed when a tariff is revised retroactively or readings arrive late
Topic	A named stream in Kafka, like a channel. Ours is meter.readings.v1
Partition	A slice of a topic. Kafka guarantees ordering within a partition, which is why we send each meter's readings to the same one
DLQ (dead letter queue)	Where records that fail validation are sent, so bad data is kept for investigation instead of silently dropped
Parquet	A compressed, column-based file format. Much smaller and faster to scan than CSV for this kind of data
Event time vs ingest time	When a reading was actually taken, versus when it reached our system. They differ when a meter is offline and catches up later
Watermark	How long the streaming job waits for late data before closing a time window and publishing the result
Idempotent	Safe to run more than once with the same result. Important because after a crash the system may reprocess data it already handled
Provisional vs settled	Provisional comes from the speed layer and may change. Settled comes from the batch layer and is final. Every figure we display says which it is
Net metering	Billing a household only on the difference between what it consumed and what its solar panels generated
Block tariff	Pricing electricity in bands, where the first N units cost less than the next N. Crossing a boundary raises the price only on the units above it
ADR	Architecture Decision Record. A short document capturing one decision, what was rejected, and what it costs
Smoke test	A quick automated check that everything is up and correctly configured, run before trusting anything else

Questions

Everything described here is in the repository at github.com/Raviyaa27/smartgrid-lambda-pipeline. The decision records in docs/adr/ go into considerably more depth than this guide, and the README covers setup. If something in this document does not make sense, please say so — if it is unclear here it will be unclear in the report too.